

per Python). The name of the module will be moulded out of the file name by leaving out the suffix `.py`. For instance, if the file name is `fibonacci.py`, the module name is actually `fibonacci`.

Here's a look at our Fibonacci functions in the form of a module at <https://dgit.in/FibPy>. All that needs to be done is that we save the code in the file named `fibonacci.py`. You can now use your newly created “fibonacci” module. Import this module for use like you would any other module in a program or a script. It should look like this: <https://dgit.in/FibDemPy>. It is not recommended to call the recursive version of something like the Fibonacci function along with large arguments. Think of a value like 42, and you realise that it is already too large. The waiting period will most likely be extremely long.

It does not take a genius to guess that it is a pain if you need to use those functions often in any of your programs. Having to type out the fully qualified name (in this case - `fibonacci.fib(7)`). A simple solution to this issue lies in assigning a local name for your module function which will help get a shorter name, as shown here: <https://dgit.in/FibLocPy>.

Usually modules are known to contain functions or classes, but it is not unheard of to see “plain” statements in them as well. These “plain” statements can be used to initialize a module. However, they will only be executed after the module is imported.

There is a private symbol table for modules and this is used as the global symbol table by all of the functions as have been defined in the module. This is a simple method of ensuring that a global variable of a module does not accidentally clash with the user's global variable having the same name. In order to access the global variables of a module, you can use the same notation as functions - `modname.name`

Also remember - a module is capable of importing other modules. It is a customary practice to place all import statements right at the beginning of a module or a script.

Types of Modules

There are different kind of modules:

- 1) Modules written in Python - They will display the suffix: `.py`
- 2) Dynamically linked C modules - Suffixes are: `.dll`, `.pyd`, `.so`, `.sl`, ...
- 3) C-Modules linked with the Interpreter: An error message is returned for Built-in-Modules.

If you have created thousands of modules over a period of time, it is easy